

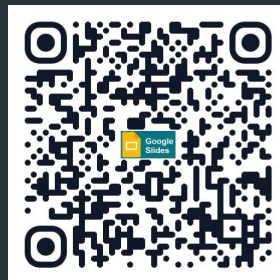


FACE RECOGNITION WITH PYTHON

A Complete AIML Course — From Fundamentals to Production Systems

NIELIT Ropar | Artificial Intelligence & Machine Learning

AIML WITH IIT ROPAR



Access the Slide:

https://docs.google.com/presentation/d/1yXeWSvg_sKOcLLEMa87chIP0A74d8VWJOkKLy2Y_wxkM/edit?usp=sharing

COURSE OVERVIEW

Three modules from basics to production-ready systems

01

Fundamentals of Face Recognition

Core concepts, detection vs. recognition, the 5-step pipeline, Python libraries, and popular models

02

Advanced Recognition & Analysis

Real-time recognition, confidence scoring, multi-face detection, DeepFace, emotion & attribute analysis

03

Multi-Person Attendance System

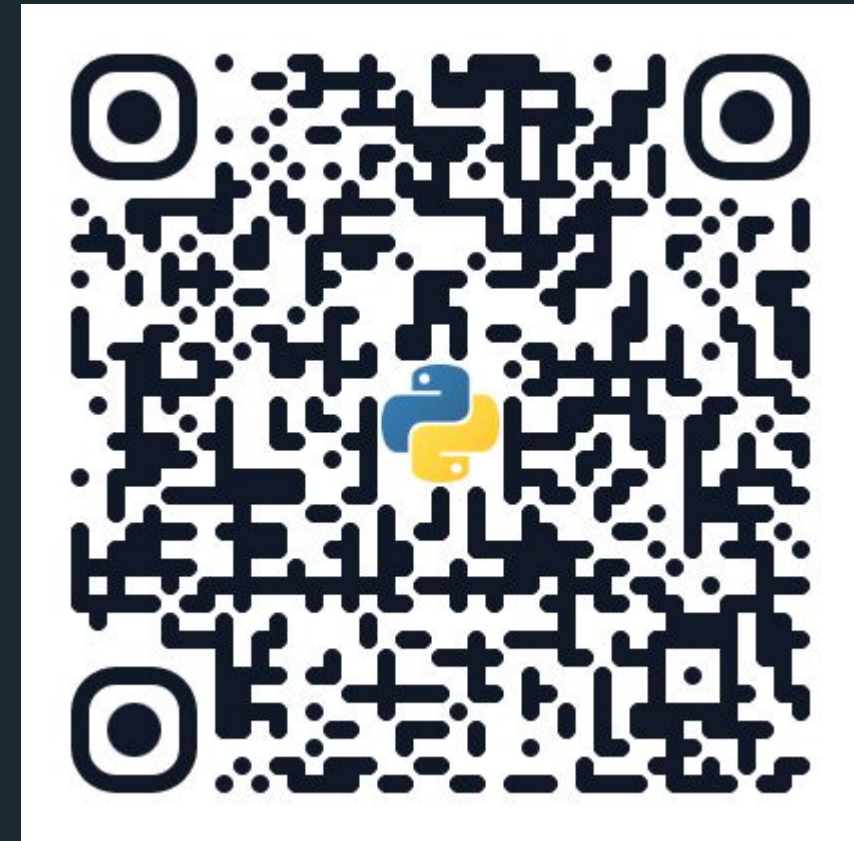
Dataset creation, face encoding database, CSV attendance logging, and duplicate prevention

MODULE 01

Fundamentals of Face Recognition

Understanding core concepts, the recognition pipeline, essential Python libraries, and building your first face recognition system

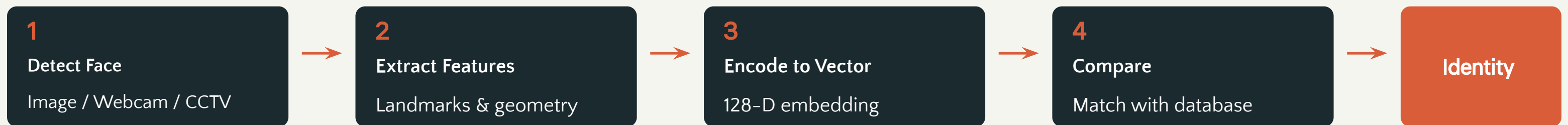
https://github.com/lovnishverma/Python-Getting-Started/blob/main/Face_Recognition_101.ipynb



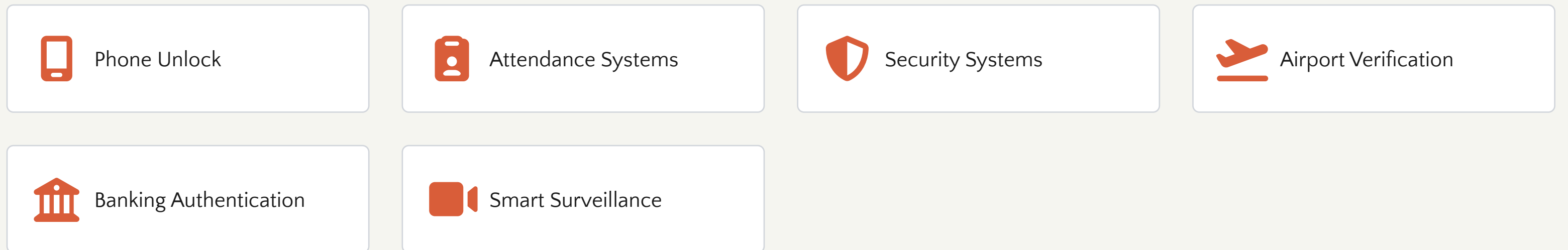
What is Face Recognition?

Face Recognition is an **AI and Computer Vision** technology that identifies or verifies a person using their face. It converts facial features into **128-dimensional numerical vectors** (face embeddings) for comparison.

The 5-Step Process



Real-World Applications



Face Detection vs. Face Recognition



Face Detection

Purpose: Finds *where* faces are located

"There is a face in this image."

Python Code

```
faces = face_recognition.face_locations(image)
```



Face Recognition

Purpose: Identifies *whose* face it is

"This is Lovnish."

Python Code

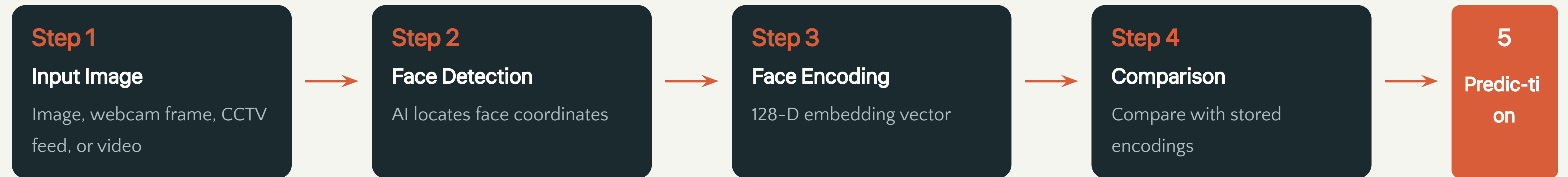
```
name = face_recognition.compare_faces(known, unknown)
```

Key Distinctions

Aspect	Face Detection	Face Recognition
Output	Bounding box coordinates	Person's identity / name
Requires Database	No	Yes
Embeddings	Not needed	128-D vector comparison
Complexity	Lower	Higher

The Face Recognition Pipeline

Five sequential steps from input image to identity prediction



Python Implementation

```
import face_recognition

# Load image and detect faces (Steps 1-2)
image = face_recognition.load_image_file("person.jpg")
locations = face_recognition.face_locations(image)

# Generate 128-D encodings (Step 3)
encodings = face_recognition.face_encodings(image, locations)

# Compare and predict (Steps 4-5)
matches = face_recognition.compare_faces(known_encodings, encodings[0])
```


Technologies and Python Libraries

LANGUAGES

Python

C++

JavaScript

KEY LIBRARIES



OpenCV

Computer vision and image processing



dlib

ML toolkit with face detection



face_recognition

Beginner-friendly, accurate, Colab-ready



DeepFace

Deep learning facial analysis



TensorFlow

Google's deep learning framework



PyTorch

Meta's deep learning framework

Quick Start — Install & Import

```
# Install
```

```
!pip install face_recognition opencv-python
```

```
# Import
```

```
import cv2; import face_recognition; import numpy as np
```



Popular Face Recognition Models

Model	Usage	Characteristics
Haar Cascade	Basic detection	Fast, older method
HOG	Fast CPU detection	Good balance speed/accuracy
CNN	Accurate DL detection	Deep learning, GPU preferred
FaceNet	Face embeddings	128-D vector generation
ArcFace	High accuracy recognition	State-of-the-art performance

Advantages

- Fast authentication in milliseconds
- Contactless verification
- Fully automated process
- Improved security over passwords

Limitations

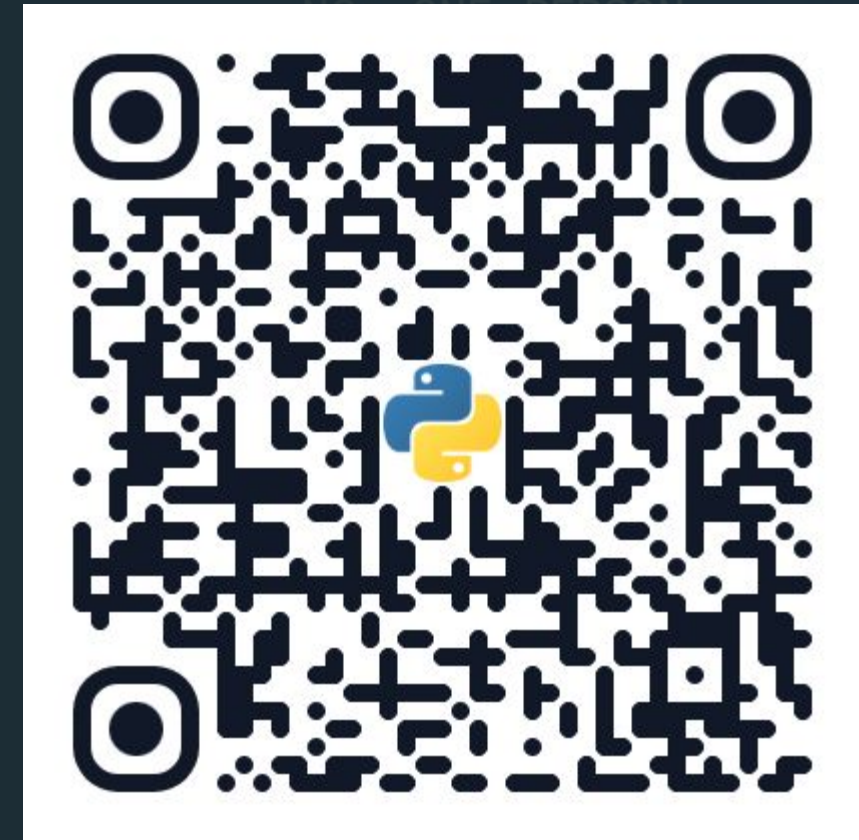
- Privacy and ethical concerns
- Sensitive to lighting changes
- Affected by pose variations
- Potential bias in datasets

MODULE 02

Advanced Recognition & Analysis

Real-time systems, confidence scoring, multi-face detection, and
DeepFace-powered facial attribute analysis

https://github.com/lovnishverma/Python-Getting-Started/blob/main/Face_Recognition_102.ipynb



Real-Time Recognition & Confidence Scores

Understanding Confidence

Face recognition uses **vector distance** — smaller distance means a better match. The confidence score is computed as a percentage from the face distance.

Confidence = (1 - face_distance) x 100%

Threshold Tuning Guide

Threshold	Accuracy	Use Case
0.40	Very Strict	High-security access
0.50	Recommended	General purpose
0.60	More Flexible	Low-security, fast
0.70	Risk of False Matches	Not recommended

Python Implementation

```
# Set threshold
THRESHOLD = 0.50

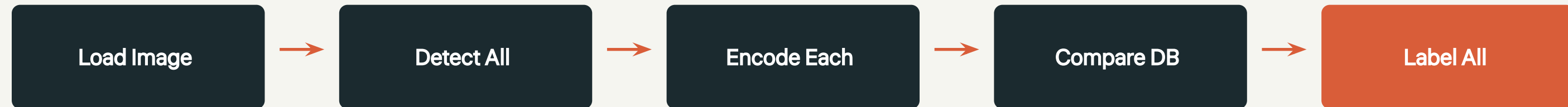
# Compare faces
matches = face_recognition.compare_faces(
    known_encodings,
    face_encoding,
    tolerance=THRESHOLD
)

# Calculate confidence
distances = face_recognition.face_distance(
    known_encodings, face_encoding
)
best = np.argmin(distances)
confidence = round((1 - distances[best]) * 100, 2)

# Display label
label = f"{name} | {confidence}%"
```

Multi-Face Recognition

Detect and recognize multiple faces in a single image or frame



```
# Load group image
group_image = face_recognition.load_image_file("group.jpg")

# Detect ALL face locations
locations = face_recognition.face_locations(group_image)
encodings = face_recognition.face_encodings(group_image, locations)

# Loop through each detected face
for (top, right, bottom, left), encoding in zip(locations, encodings):
    matches = face_recognition.compare_faces(known_encodings, encoding, tolerance=0.5)
    distances = face_recognition.face_distance(known_encodings, encoding)
    best_index = np.argmin(distances)
    name = known_names[best_index] if matches[best_index] else "Unknown"
    cv2.rectangle(image, (left, top), (right, bottom), (0, 255, 0), 3)
    cv2.putText(image, name, (left, top-10), font, 0.8, (255, 0, 0), 2)
```

DeepFace: Advanced Facial Analysis

Higher-level deep learning library for comprehensive facial analysis

DeepFace Capabilities



Verification



Recognition



Emotion



Age



Gender



Race

Analyze Face Attributes

```
from deepface import DeepFace
# Analyze image
result = DeepFace.analyze(
    img_path="photo.jpg",
    actions=['emotion',
            'age',
            'gender'],
    enforce_detection=False
)
```

Output Results

Accessing Results:

- **Age:** result['age']
- **Gender:** result['dominant_gender']
- **Emotion:** result['dominant_emotion']

Emotion Categories:

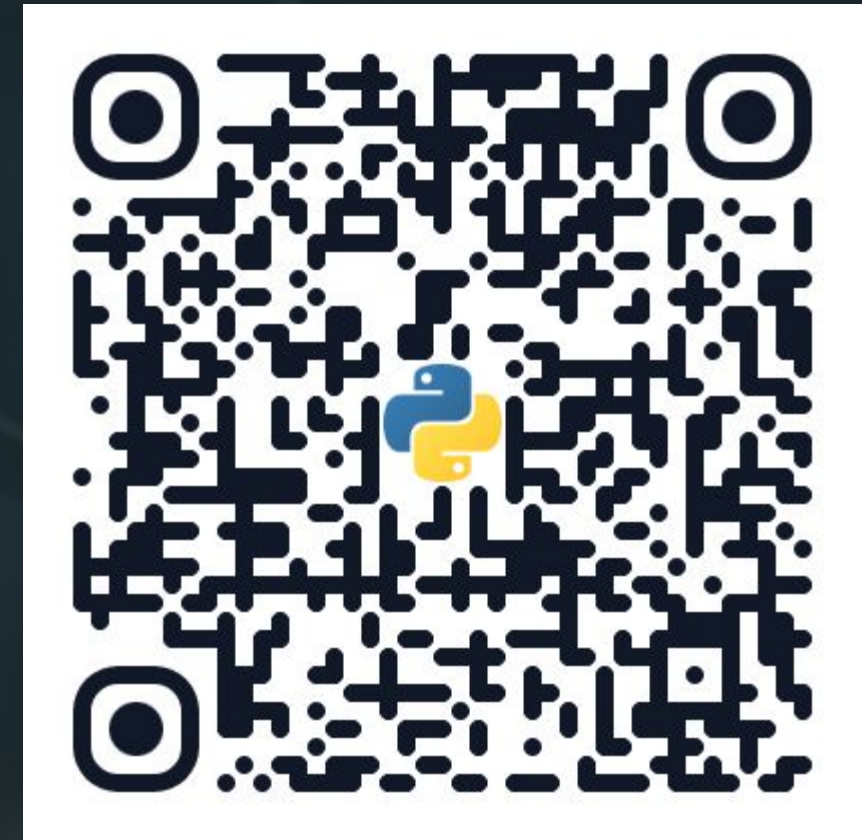
angry, disgust, fear, happy,
sad, surprise, neutral

MODULE 03

Multi-Person Attendance System

Building a complete production-ready AI attendance system with dataset management and CSV export

https://github.com/lovnishverma/Python-Getting-Started/blob/main/Face_Recognition_103.ipynb



Building the Face Database

Organized dataset creation with multiple images per person for robust encoding

Dataset Folder Structure

```
face_attendance_dataset/  
lovnish/  
lovnish_1.jpg  
lovnish_2.jpg  
lovnish_3.jpg  
rahul/  
rahul_1.jpg  
rahul_2.jpg
```

Input Methods



Webcam

Capture 3-5 photos per person via browser



Upload

Manually upload existing face images

Building the Encoding DB

```
known_encodings = []  
known_names = []  
  
# Iterate dataset folder  
for person in os.listdir(DATASET):  
    person_path = DATASET / person  
    for img_name in os.listdir(person_path):  
        img = face_recognition.load_image_file(  
            person_path / img_name)  
        encs = face_recognition.face_encodings(img)  
        if encs:  
            known_encodings.append(encs[0])  
            known_names.append(person)  
    print(f"Total: {len(known_names)} faces")
```



Best Practice: Use 3-5 clear frontal images per person with good lighting. The system handles "no face found" errors gracefully by skipping invalid images.

Attendance System with CSV Export

Complete Workflow



Attendance Logging Function

```
# Duplicate prevention
marked_names = set()
def mark_attendance(name, conf):
    if name in marked_names:
        return
    marked_names.add(name)
    date = time.strftime("%Y-%m-%d")
    tm = time.strftime("%H:%M:%S")
    with open(CSV, "a") as f:
        writer = csv.writer(f)
        writer.writerow([name, conf, date, tm])
```

CSV Output Format

Name	Confidence	Date	Time
Lovnish	92.5%	2025-03-15	09:30:22
Rahul	88.3%	2025-03-15	09:30:24
Priya	95.1%	2025-03-15	09:30:25

System Features

- Duplicate detection prevents re-marking
- Confidence score for each entry
- Automatic date & timestamp
- Export to CSV for Excel analysis

Best Practices, Ethics & Learning Roadmap

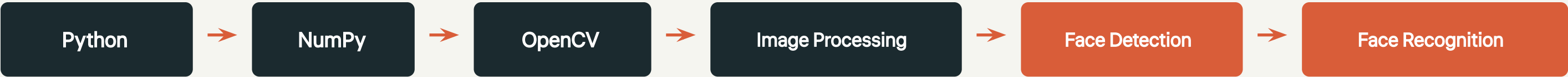
Technical Best Practices

- Use clear frontal face images
- Capture 3-5 images per person
- Maintain proper, even lighting
- Avoid blurry or low-res images
- Tune threshold between 0.4-0.6
- Store embeddings securely

Ethical Considerations

- Always obtain user consent
- Protect stored face data (encryption)
- Avoid surveillance misuse
- Follow privacy laws (GDPR, IT Act)
- Be transparent about data usage
- Audit for bias in recognition

Learning Roadmap



Recommended Projects



Smart Attendance System



Face Unlock App



AI Door Lock System



Visitor Counter



Classroom Monitoring



Flask Web App

START BUILDING WITH FACE RECOGNITION

You now have the skills to build production-grade AI face recognition systems.

From attendance automation to security — the possibilities are endless.

SKILLS YOU HAVE MASTERED

Face Detection Pipeline

128-D Embeddings

Multi-Face Recognition

DeepFace Analysis

Confidence Scoring

CSV Attendance System

Threshold Tuning

Build your first project today!